Serveur web en C : les bases

Oct 25, 2026 · @angeli

Objectif : toi, tu es Tomcat et la servlet

En Java, Tomcat gère le réseau et appelle ta méthode `doGet(req, resp)`. En C, il n'y a pas de Tomcat : tu écris le conteneur et la servlet. Ce document explique les concepts, donne des squelettes, et propose une roadmap d'exercices dont la finalité est un serveur HTTPS en C. Le code complet, c'est à toi de l'écrire.

Ce que fait Java (servlet)	Ce que tu fais en C
Tomcat ouvre le port 8080	`socket()`, `bind()`, `listen()`
Tomcat attend un client	`accept()`
Tomcat transforme le texte reçu en `HttpServletRequest`	tu lis les octets avec `read()` et tu découpes la première ligne
`@WebServlet("/hello")` ou `web.xml`	une table de routes ou un `strcmp(path, "/hello")`
`resp.setContentType("text/html")`	tu écris toi-même l'en-tête `Content-Type: text/html`
`resp.getWriter().write("...")`	tu accumules le texte, puis `write(fd, ...)`
Tomcat ferme la connexion	`close(fd)`

Comment marche une app web

Une app web, c'est un échange de texte : le client demande, le serveur répond. Le serveur ne parle jamais en premier.

- **Client** : le navigateur (ou `curl`, ou une app mobile).
- **Serveur** : ton programme C, qui attend sur un **port** (ex. 8080) d'une machine identifiée par une **adresse IP** (`localhost` = `127.0.0.1` = ta propre machine).

- **TCP** : le tuyau entre les deux. Il garantit que les octets arrivent dans l'ordre, sans perte. Mais c'est un **flux** : il ne connaît pas la notion de « message ».
- **HTTP** : la langue parlée dans ce tuyau. C'est du texte lisible, avec des règles précises (section suivante).

Ce qui se passe quand tu tapes `http://localhost:8080/hello` :

1. Le navigateur trouve l'IP de `localhost`.
2. Il ouvre une connexion TCP vers le port 8080.
3. Il envoie une requête HTTP en texte.
4. Ton serveur lit ce texte, décide quoi faire selon le chemin `/hello`, et écrit une réponse.
5. Le navigateur affiche le HTML reçu. S'il y voit `<link href="style.css">` ou `<script src="app.js">`, il refait **une requête par fichier**.

Une « app web » complète a trois rôles, tous servis par le même mécanisme :

- **Front** : HTML, CSS et JS que le serveur envoie et que le navigateur exécute.
- **API** : des chemins (ex. `/api/users`) qui renvoient des données, souvent en JSON, au lieu d'une page.
- **Le lien entre les deux** : le JavaScript de la page appelle l'API avec `fetch()` et met la page à jour.

HTTP est **sans état** : chaque requête est indépendante. Pour « se souvenir » d'un utilisateur (connexion, panier), on utilise des **cookies**, comme `HttpSession` en Java.

Le protocole HTTP

Une requête et une réponse ont la même forme : une première ligne, des en-têtes, une ligne vide, puis un corps optionnel. Chaque fin de ligne est `\r\n` (CRLF), pas `\n` seul.

Ce que le navigateur t'envoie :

```
GET /hello?nom=Ando HTTP/1.1
Host: localhost:8080
User-Agent: Mozilla/5.0 ...
Accept: text/html
```

- **Ligne de requête** : méthode (`GET`), chemin (`/hello?nom=Ando`), version (`HTTP/1.1`). Tout ce qui suit `?` est la **query string**, l'équivalent de `req.getParameter("nom")`.
- **En-têtes** : une ligne `Nom: valeur` chacun.

- **Ligne vide** (`\r\n\r\n` au total) : fin des en-têtes. C'est ce que tu cherches pour savoir que la requête est complète.
- **Corps** : seulement pour `POST` / `PUT`. Sa taille est donnée par l'en-tête `Content-Length`.

Ce que tu dois renvoyer :

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 42
Connection: close

<html><body><h1>Bonjour</h1></body></html>
```

`Content-Length` doit être le nombre exact d'**octets** du corps (ici 42). S'il est faux, le navigateur attend des octets qui ne viennent pas, ou coupe la page.

Les méthodes principales : `GET` lit, `POST` envoie ou crée, `PUT` remplace, `DELETE` supprime.

Code	Sens	Quand l'utiliser
200 OK	succès	la page ou les données sont dans le corps
302 Found	redirection	avec l'en-tête `Location: /autre-page`
400 Bad Request	requête mal formée	première ligne illisible
404 Not Found	chemin inconnu	aucune route ne correspond
405 Method Not Allowed	mauvaise méthode	`POST` sur une route qui n'accepte que `GET`
500 Internal Server Error	erreur chez toi	ton code a échoué (fichier introuvable, etc.)

Pour voir l'échange brut : `curl -v http://localhost:8080/hello` affiche la requête (lignes `>`) et la réponse (lignes `<`).

Le cycle de vie d'un serveur en C

Un serveur, c'est trois appels de préparation, puis une boucle infinie qui traite un client à chaque tour. Un socket est un **descripteur de fichier** (un `int`), comme un fichier ouvert : on y fait `read()` et `write()`.

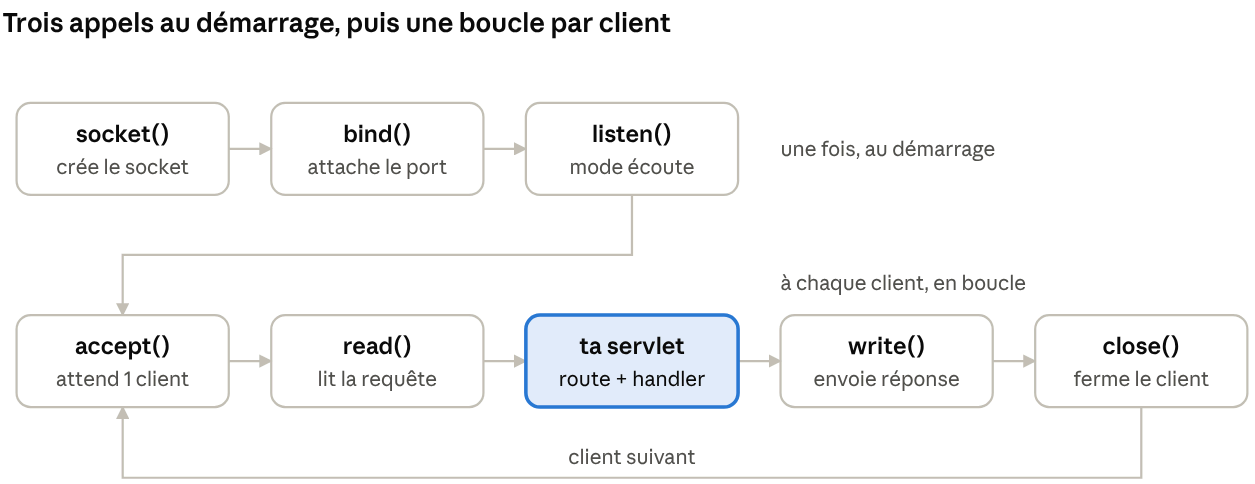
En-têtes à inclure : `<sys/socket.h>`, `<netinet/in.h>`, `<arpa/inet.h>`, `<unistd.h>`. Ces appels sont POSIX : ils marchent sur Linux et macOS, pas tels quels sous Windows (Winsock).

Préparation, une seule fois :

1. `socket(AF_INET, SOCK_STREAM, 0)` crée le socket. `AF_INET` = IPv4, `SOCK_STREAM` = TCP.
2. `setsockopt(..., SOL_SOCKET, SO_REUSEADDR, ...)` permet de relancer le serveur tout de suite après l'avoir arrêté, sans l'erreur « Address already in use ».
3. `bind()` attache le socket à un port. Tu remplis une `struct sockaddr_in` ; le port passe par `htons(8080)`, qui le convertit dans l'ordre d'octets du réseau.
4. `listen(fd, 16)` rend le socket passif : il attend des clients. 16 = taille de la file d'attente.

Boucle, à chaque client :

1. `accept()` bloque jusqu'à ce qu'un client arrive, puis renvoie un **nouveau** descripteur, propre à ce client. Le premier continue d'écouter.
2. `read()` lit les octets de la requête. Attention : TCP est un flux, un seul `read()` peut ne renvoyer qu'une partie de la requête.
3. Ton code découpe la requête, choisit la route et prépare la réponse.
4. `write()` envoie la réponse. Il peut écrire moins que demandé : vérifie sa valeur de retour.
5. `close()` ferme le descripteur du client. Oublie-le, et ton programme finira par manquer de descripteurs.



boucle d'un serveur HTTP en C · 3 appels + 5 étapes par client

Seule la case colorée est « ton » code métier ; tout le reste est ce que Tomcat faisait pour toi.

Point clé : il y a **deux** descripteurs. Celui d'écoute (créé une fois) et celui du client (un par `accept()`). Chaque fonction renvoie `-1` en cas d'erreur ; `perror("bind")` t'affiche pourquoi.

Pour les détails de chaque fonction : `man 2 socket`, `man 2 bind`, `man 2 accept`, `man 7 ip`.

Servir une page comme `resp.getWriter().write`

Pour écrire comme en Java, crée ton propre « objet réponse » : une `struct` qui **accumule** le corps, puis une fonction qui envoie tout d'un coup. On accumule parce que `Content-Length` doit partir **avant** le corps, donc il faut connaître sa taille d'abord. Tomcat fait pareil : il garde ce que tu écris dans un tampon.

Le squelette (les corps des fonctions sont à écrire par toi) :

```
typedef struct {
    int         fd;           // socket du client (renvoyé par accept)
    int         status;       // 200, 404...
    const char *type;         // "text/html; charset=utf-8"
    char        body[65536];  // corps accumulé
    size_t      len;          // octets déjà dans body
} Response;

void resp_write(Response *r, const char *s); // ≈ resp.getWriter().write(s)
void resp_send(Response *r);                 // en-têtes + corps sur r->fd
```

Ta « servlet » ressemble alors à du Java :

```
void hello_get(Request *req, Response *resp) {
    resp->type = "text/html; charset=utf-8";
    resp_write(resp, "<html><body>");
    resp_write(resp, "<h1>Bonjour</h1>");
    resp_write(resp, "</body></html>");
}
```

Pour remplacer `web.xml`, une table de routes avec des **pointeurs de fonction** :

```
typedef void (*Handler)(Request *, Response *);
typedef struct { const char *method; const char *path; Handler fn; } Route;

Route routes[] = {
    { "GET", "/hello", hello_get },
};
```

Ce que tu dois écrire, avec des pistes :

- `Request` : à toi de choisir ses champs. Au minimum `method` et `path` ; plus tard `query`, les en-têtes, le corps. Remplis-la depuis la première ligne de la requête (`sscanf` avec une largeur maximale, ex. `%7s %255s`, ou `strtok`).
- `resp_write` : copie `s` à la fin de `body` (`memcpy`) et augmente `len`. **Vérifie** qu'il reste de la place, sinon dépassement de tampon.
- `resp_send` : construis la ligne de statut et les en-têtes avec `snprintf` (le texte du statut dépend du code : `200` → `OK`, `404` → `Not Found`), puis `write()` les en-têtes, puis `write()` le corps.
- La boucle principale : pour chaque client, remplir `Request`, initialiser `Response` (status 200, len 0), parcourir `routes[]`, appeler `fn`, sinon 404, puis `resp_send` et `close`.

HTTPS en C

HTTPS, c'est le même HTTP, transporté dans un tunnel chiffré appelé **TLS**. Ton parsing, tes routes et ta `Response` ne changent pas : seules la préparation, la lecture et l'écriture sur le socket changent. On n'écrit jamais le chiffrement soi-même, on utilise une bibliothèque (OpenSSL dans ce document).

Les concepts à comprendre :

- **Clé privée** (`key.pem`) : un secret que seul ton serveur possède. Elle ne doit jamais être partagée ni publiée.
- **Certificat** (`cert.pem`) : une carte d'identité publique. Il dit « ce serveur est `localhost` » et contient la clé publique qui va avec ta clé privée.
- **Autorité de certification (CA)** : un organisme qui signe les certificats. Le navigateur fait confiance à une liste de CA. Un certificat **auto-signé** est signé par lui-même : le navigateur affiche un avertissement, ce qui est normal pour des tests.
- **SAN (Subject Alternative Name)** : la liste des noms pour lesquels le certificat est valable. Les navigateurs modernes vérifient ce champ, pas seulement le nom principal (`CN`).
- **Handshake** : la négociation au début de chaque connexion. Le serveur présente son certificat, puis client et serveur s'accordent sur une clé de chiffrement.

Ce qui change dans ta boucle serveur :

Étape	HTTP	HTTPS avec OpenSSL
Démarrage	rien	`SSL_CTX_new(TLS_server_method())` , puis `SSL_CTX_use_certificate_file` et `SSL_CTX_use_PrivateKey_file`
Nouveau client	`accept()`	`accept()` , puis `SSL_new` , `SSL_set_fd` , `SSL_accept` (le handshake)
Lire	`read(fd, ...)`	`SSL_read(ssl, ...)`
Écrire	`write(fd, ...)`	`SSL_write(ssl, ...)`
Fin	`close(fd)`	`SSL_shutdown` , `SSL_free` , puis `close(fd)`
Erreurs	`perror`	`ERR_print_errors_fp(stderr)`

Conseil d'architecture : si tout ton code lit et écrit à travers **une seule paire de fonctions** (par exemple une struct `Conn` avec deux pointeurs de fonction `read` et `write`), passer de HTTP à HTTPS ne touche que ces deux fonctions.

Par convention, HTTPS écoute sur le port 443 (8443 pour tes tests, sans droits root). Une connexion `http://` vers un port HTTPS échoue : le serveur attend un handshake, pas du texte.

Créer un certificat auto-signé

Le but est d'obtenir deux fichiers au format PEM : `cert.pem` (le certificat) et `key.pem` (la clé privée). C'est le format que lit OpenSSL.

Sous Ubuntu

Installer l'outil, puis générer clé et certificat en une commande :

```
sudo apt install openssl

openssl req -x509 -newkey rsa:2048 -nodes \
  -keyout key.pem -out cert.pem -days 365 \
  -subj "/CN=localhost" \
  -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
```

Option	Rôle
`req -x509`	produit directement un certificat auto-signé, pas une demande de signature
`-newkey rsa:2048`	génère en même temps une nouvelle clé RSA de 2048 bits
`-nodes`	ne chiffre pas la clé avec un mot de passe, sinon ton serveur le demanderait au démarrage
`-days 365`	durée de validité
`-subj "/CN=localhost"`	le nom principal, sans questions interactives
`-addext "subjectAltName=..."`	les noms valables (SAN), exigés par les navigateurs modernes

Vérifier ce que contient le certificat :

```
openssl x509 -in cert.pem -noout -subject -enddate -ext subjectAltName
```

Ces commandes ont été testées ici avec OpenSSL 3.0.13 : le certificat obtenu est accepté par `curl --cacert cert.pem` et par `openssl s_client` (TLS 1.3, « Verify return code: 0 (ok) »).

Sous Windows

Méthode recommandée : OpenSSL via MSYS2. MSYS2 te donne aussi gcc et la bibliothèque OpenSSL pour compiler (voir la section suivante). Dans le terminal **MSYS2 UCRT64**, la commande est la même que sous Ubuntu, avec une différence : écris `-subj "//CN=localhost"` (deux barres). Le terminal MSYS2, comme Git Bash, transforme sinon `/CN=...` en chemin de fichier Windows.

Méthode PowerShell, sans rien installer pour créer le certificat. Windows crée le certificat dans son magasin, il faut ensuite l'exporter puis le convertir :

```
$cert = New-SelfSignedCertificate -DnsName "localhost" -CertStoreLocation
"cert:\CurrentUser\My"
$pwd  = ConvertTo-SecureString -String "motdepasse" -Force -AsPlainText
Export-PfxCertificate -Cert ("cert:\CurrentUser\My\" + $cert.Thumbprint) -
FilePath .\cert.pfx -Password $pwd
```

Le fichier `.pfx` contient certificat et clé, mais OpenSSL attend du PEM. La conversion demande quand même l'outil `openssl` (MSYS2 ou Git Bash) :

```
openssl pkcs12 -in cert.pfx -clcerts -nokeys -out cert.pem
openssl pkcs12 -in cert.pfx -nocerts -nodes -out key.pem
```

Les deux commandes de conversion ont été testées ici sur un `.pfx` produit par OpenSSL. Les commandes Windows (PowerShell, MSYS2) n'ont **pas** pu être testées : cet environnement est sous Linux. Si `openssl pkcs12` refuse un `.pfx` exporté par Windows avec une erreur « unsupported », ajoute l'option `-legacy`.

Dans tous les cas : mets `key.pem` dans ton `.gitignore`, ne le publie jamais.

Bibliothèques et outils

Tout le projet tient avec deux bibliothèques : les sockets du système et OpenSSL. Le reste est optionnel.

Bibliothèque	Rôle	Ubuntu	Windows (MSYS2 UCRT64)	Option gcc
Sockets POSIX (libc)	réseau : `socket`, `accept`, `read`…	incluse	non disponible, voir Winsock	aucune
Winsock 2	réseau sous Windows	sans objet	incluse dans Windows	`-lws2_32`
OpenSSL 3 (`libssl`, `libcrypto`)	TLS pour HTTPS	`libssl-dev`	`mingw-w64-ucrt-x86_64-openssl`	`-lssl -lcrypto`
pthread (optionnel)	plusieurs clients en parallèle	incluse	incluse avec gcc	`-pthread`
cJSON (optionnel)	produire et lire du JSON pour l'API	`libcjson-dev`	à compiler ou copier dans ton projet	`-lcjson`

Installation complète sous Ubuntu :

```
sudo apt install build-essential libssl-dev openssl curl
```

Installation sous Windows : installe MSYS2, ouvre le terminal **UCRT64**, puis :

```
pacman -S mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-openssl
```

Sous Windows, le code réseau diffère de Linux sur quelques points : appeler `WSAStartup()` au démarrage, `closesocket()` au lieu de `close()`, `recv()` / `send()` au lieu de `read()` / `write()`, et l'en-tête `<winsock2.h>`. Le reste (OpenSSL, HTTP, routes) est identique. Pour apprendre, le plus simple sous Windows reste **WSL** (un Ubuntu dans Windows) : tout ce document s'y applique tel quel.

Alternatives à OpenSSL, si tu veux plus léger : **mbedTLS** et **wolfSSL**, conçus pour l'embarqué. Leur API est différente, mais les concepts (contexte, handshake, lecture, écriture) sont les mêmes.

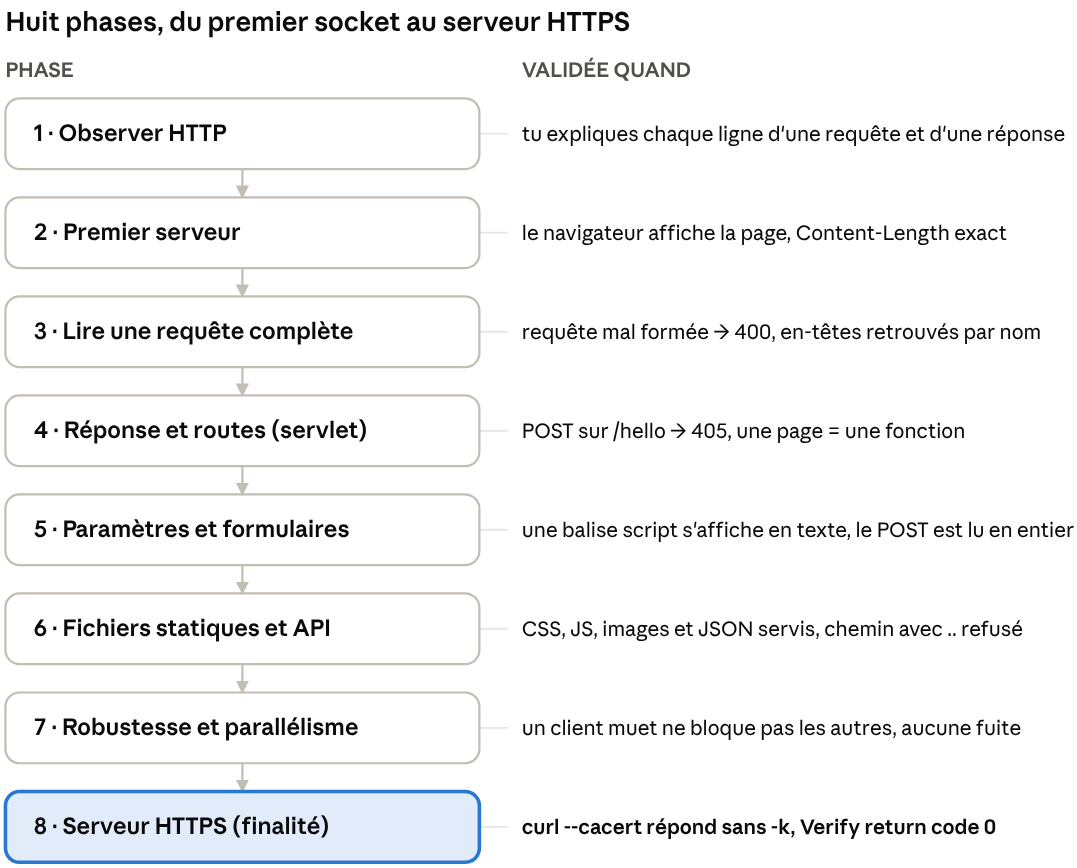
Outils de test :

- `curl -v` : voir la requête et la réponse ; `curl --cacert cert.pem` pour faire confiance à ton certificat auto-signé.
- `openssl s_client -connect localhost:8443` : voir le handshake TLS et le certificat présenté.
- `nc` (netcat) : envoyer du texte brut à la main.
- `valgrind` ou l'option gcc `-fsanitize=address` : trouver les fuites et les accès mémoire invalides (sous Linux).

Source du nom de paquet MSYS2 : packages.msys2.org.

Roadmap : du socket au serveur HTTPS

Huit phases, chacune ajoute un concept et se termine par un test qui prouve qu'elle est finie. Ne passe pas à la suivante tant que le test de validation ne réussit pas. Les exercices disent **quoi** faire et **où chercher**, pas comment l'écrire.



roadmap · 8 phases et leur critère de validation

Chaque phase réutilise le code de la précédente : à la phase 8, seules les fonctions de lecture et d'écriture changent.

Phase 1 : observer HTTP

- [ ] Lance `nc -l 8080` (ou `nc -l -p 8080`), ouvre `http://localhost:8080` dans le navigateur, et note le rôle de chaque ligne reçue.
- [ ] Dans le même terminal, tape une réponse HTTP complète à la main et regarde le navigateur l'afficher.
- [ ] Lance `curl -v http://example.com` et repère la ligne de statut et les en-têtes.

Validé quand tu sais expliquer chaque ligne d'une requête et d'une réponse sans regarder ce document.

Phase 2 : premier serveur

- [ ] Avec `socket`, `bind`, `listen`, `accept`, écris un serveur qui répond la même page fixe à toute requête, puis ferme la connexion et attend le client suivant.
- [ ] Vérifie la valeur de retour de chaque appel et affiche l'erreur avec `perror`.
- [ ] Calcule `Content-Length` à partir du corps, jamais à la main.

À chercher : `man 2 socket`, `man 2 bind`, `htons`, `struct sockaddr_in`, `SO_REUSEADDR`.

Validé quand le navigateur affiche la page, `curl -v` montre un `Content-Length` égal à la taille du corps, et relancer le serveur aussitôt ne donne pas « Address already in use ».

Phase 3 : lire une requête complète

- [ ] Lis en boucle jusqu'à trouver `\r\n\r\n`, avec une taille maximale (au-delà : statut 431).
- [ ] Découpe la première ligne en méthode, chemin et version. Sépare la query string du chemin.
- [ ] Range les en-têtes dans ta `struct Request`, et écris une fonction qui retrouve un en-tête par son nom sans tenir compte des majuscules.
- [ ] Réponds 400 si la première ligne est mal formée.

À chercher : `strstr`, `strcasecmp`, `strtok_r` ou `strchr`.

Validé quand `printf 'n_importe_quoi\r\n\r\n' | nc localhost 8080` renvoie 400, et `curl -H "X-Test: 42" localhost:8080` fait afficher `42` dans la console du serveur.

Phase 4 : l'objet réponse et les routes (ta « servlet »)

- [ ] Écris `Response`, `resp_write` et `resp_send` (section « Servir une page »).
- [ ] Écris la table de routes avec pointeurs de fonction : 404 si aucun chemin ne correspond, 405 si le chemin existe mais pas la méthode (avec l'en-tête `Allow`).
- [ ] Encapsule : `Response` en type opaque dans `response.h` / `response.c`, avec un constructeur `resp_new` et un destructeur `resp_free`.

Validé quand `curl -X POST localhost:8080/hello` renvoie 405, et ajouter une page ne demande qu'une fonction et une ligne dans la table.

Phase 5 : paramètres et formulaires

- [ ] Lis les paramètres de la query string (`?nom=Ando&age=20`).
- [ ] Décode l'encodage d'URL : `%20` et `+` deviennent des espaces.
- [ ] Échappe `<`, `>`, `&` et `"` avant d'écrire une valeur dans le HTML.
- [ ] Ajoute un formulaire HTML en `method="post"` : lis exactement `Content-Length` octets de corps. Son format est le même que la query string.

Validé quand le nom `<script>alert(1)</script>` s'affiche en texte sans s'exécuter, et `curl -d "nom=Ando" localhost:8080/form` renvoie la bonne page.

Phase 6 : fichiers statiques et API JSON

- [ ] Sers le dossier `public/`, avec le `Content-Type` choisi selon l'extension (`.html`, `.css`, `.js`, `.png`, `.json`).
- [ ] Gère les fichiers binaires (images) : ouverture en `"rb"`, taille connue avant l'envoi, envoi par morceaux.
- [ ] Refuse tout chemin qui contient `..`.
- [ ] Ajoute une route `/api/...` qui renvoie du JSON, et une page dont le JavaScript l'appelle avec `fetch`.

À chercher : `fopen`, `fread`, `stat` (`man 2 stat`).

Validé quand une page avec CSS, JS et image s'affiche entièrement, la page montre les données de l'API, et `curl --path-as-is localhost:8080/../server.c` est refusé (sans `--path-as-is`, curl retire lui-même le `..`).

Phase 7 : robustesse et plusieurs clients

- [ ] Écris en boucle jusqu'à ce que tout soit envoyé (`write` peut écrire moins que demandé).
- [ ] Ignore `SIGPIPE` .
- [ ] Mets un délai maximal de lecture, pour qu'un client muet ne bloque pas ton serveur.
- [ ] Traite plusieurs clients en parallèle : d'abord avec `fork()` , puis avec des threads, et compare.
- [ ] Journalise une ligne par requête : méthode, chemin, statut.
- [ ] Prépare HTTPS : fais passer toutes les lectures et écritures par une struct `Conn` à deux pointeurs de fonction (`read` , `write`).

À chercher : `SO_RCVTIMEO` , `man 2 fork` , `pthread_create` , `waitpid` .

Validé quand un `nc localhost 8080` laissé ouvert ne bloque pas un `curl` lancé en même temps, et `-fsanitize=address` ne signale rien après 1 000 requêtes (`for i in $(seq 1000); do curl -s localhost:8080/ > /dev/null; done`).

Phase 8 : serveur HTTPS (la finalité)

- [ ] Génère `cert.pem` et `key.pem` (section « Créer un certificat auto-signé »).
- [ ] Au démarrage, crée le contexte OpenSSL, charge le certificat et la clé, et vérifie qu'ils vont ensemble.
- [ ] Après chaque `accept` , fais le handshake. Un handshake raté ferme ce client, pas tout le serveur.
- [ ] Écris la version TLS des fonctions `read` et `write` de ta struct `Conn` . Le reste de ton code ne doit pas changer.
- [ ] Termine proprement chaque connexion TLS avant de fermer le socket.
- [ ] Bonus : refuse les versions de TLS antérieures à 1.2.
- [ ] Bonus : un second serveur HTTP sur 8080 qui redirige tout vers `https://localhost:8443` (statut 301, en-tête `Location`).

À chercher : `man SSL_CTX_new` , `man SSL_CTX_use_certificate_file` , `man SSL_CTX_check_private_key` , `man SSL_accept` , `man SSL_read` , `man SSL_CTX_set_min_proto_version` (pages installées avec `libssl-dev`).

Validé quand `curl --cacert cert.pem https://localhost:8443/` répond sans l'option `-k` , `openssl s_client -connect localhost:8443 -CAfile cert.pem` affiche « Verify return code: 0 (ok) », le navigateur affiche ta page après l'avertissement, et `curl`

`http://localhost:8443/` échoue sans faire tomber le serveur.

Pièges fréquents

- **`\n` au lieu de `\r\n`** dans les en-têtes : certains clients l'acceptent, d'autres non. Respecte la norme.
- **`Content-Length` faux** : la page reste en chargement ou est coupée. Utilise `strlen` ou ton compteur `len`, jamais un nombre écrit à la main.
- **Lecture partielle** : un `read()` ne garantit pas la requête entière. Relis jusqu'à trouver `\r\n\r\n`.
- **Le client ferme pendant ton `write()`** : Linux envoie le signal `SIGPIPE`, qui tue ton serveur par défaut. Ignore-le avec `signal(SIGPIPE, SIG_IGN);` au démarrage.
- **`sscanf("%s")` sans largeur** : dépassement de tampon. Toujours `%255s` pour un tableau de 256.
- **Port inférieur à 1024** (ex. 80) : il faut les droits root. Reste sur 8080 pour apprendre.
- **Deux requêtes pour une page** : le navigateur demande aussi `/favicon.ico`. C'est normal, ton 404 suffit.

Pour aller plus loin

Une fois la roadmap terminée, tu as un serveur HTTPS complet. Chaque étape suivante réutilise le même mécanisme.

- **API JSON** : une route comme les autres, avec `Content-Type: application/json` et un corps JSON (`snprintf` au début, une bibliothèque comme cJSON ensuite).
- **POST** : après la ligne vide, lis exactement `Content-Length` octets de corps.
- **Plusieurs clients à la fois** : ton serveur traite un client à la fois. Les options, de la plus simple à la plus performante : `fork()` par client, un thread par client (`pthread`), ou `poll()` / `epoll()`.
- **Sessions** : envoie `Set-Cookie: session=...`, relis l'en-tête `Cookie` aux requêtes suivantes.
- **HTTPS** : en production, on place souvent un nginx devant son serveur pour gérer les certificats publics (Let's Encrypt).

Références :

- Beej's Guide to Network Programming : le guide de référence sur les sockets en C.

- RFC 9110 (sens de HTTP) et RFC 9112 (format des messages HTTP/1.1).
- Les pages `man` citées plus haut.